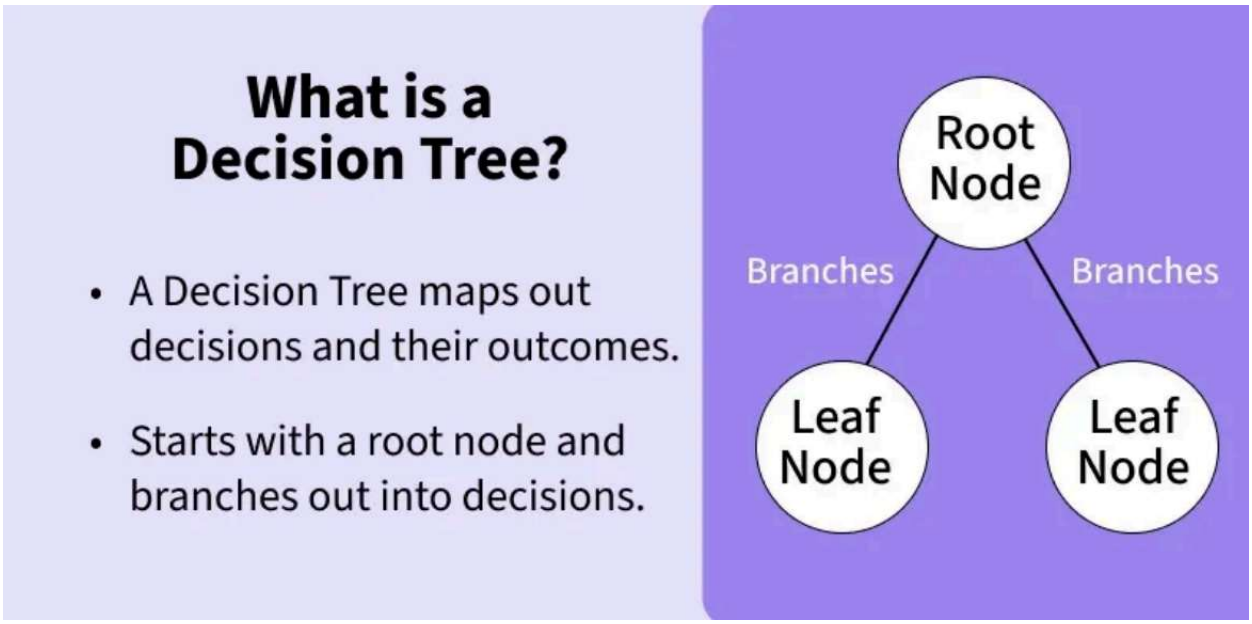
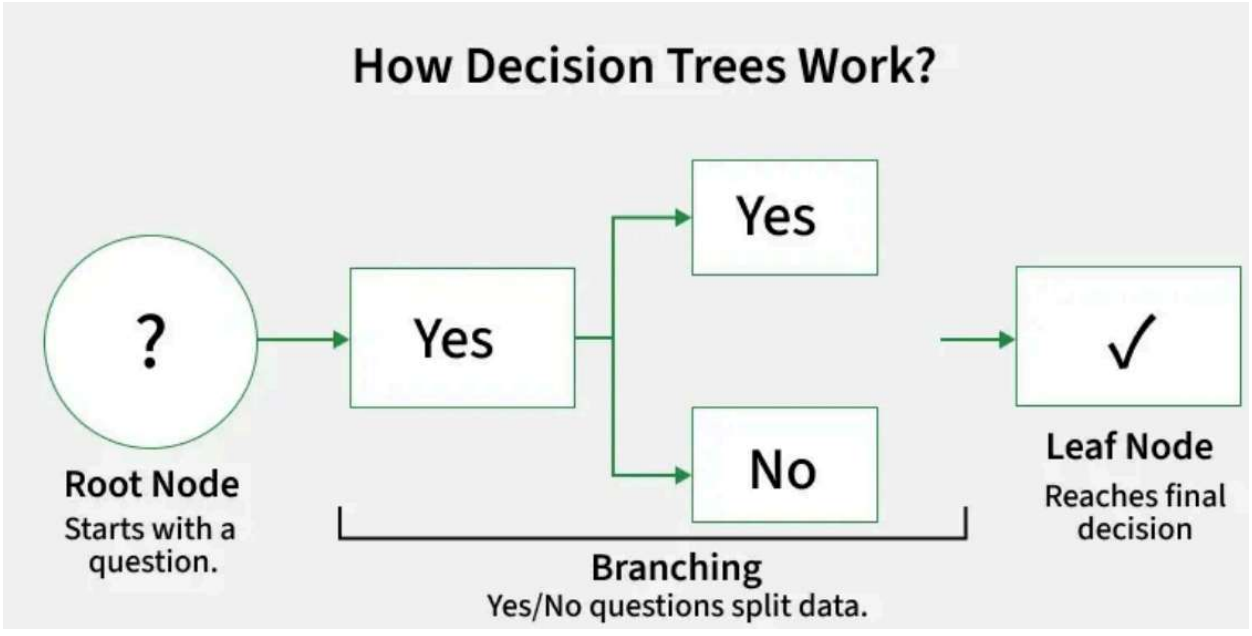
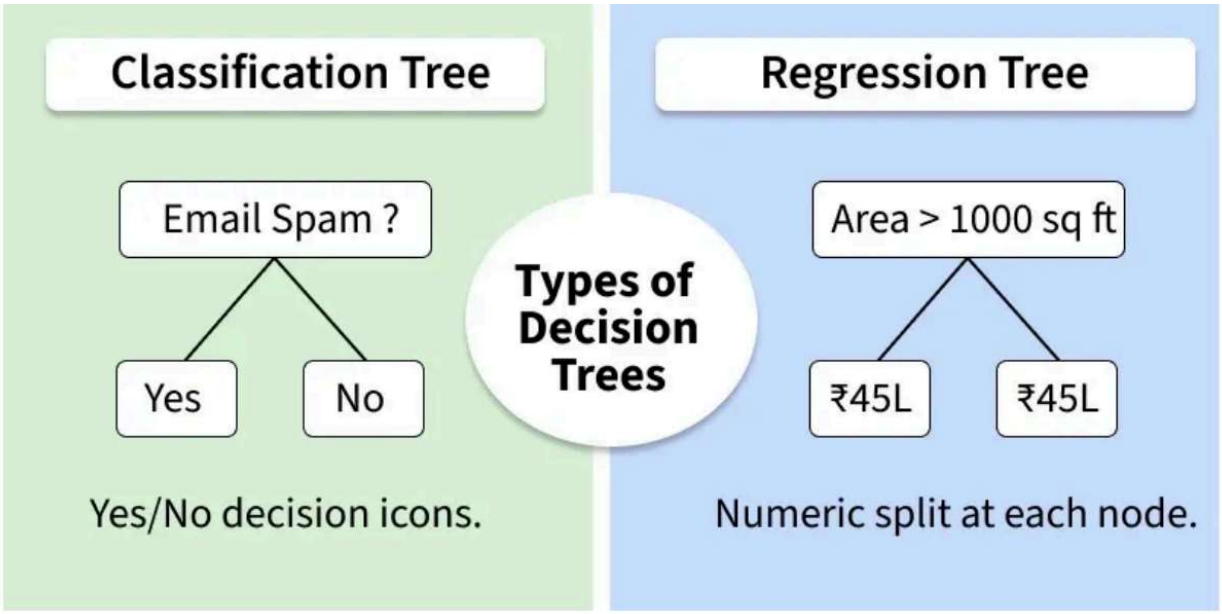
DECISION TREE

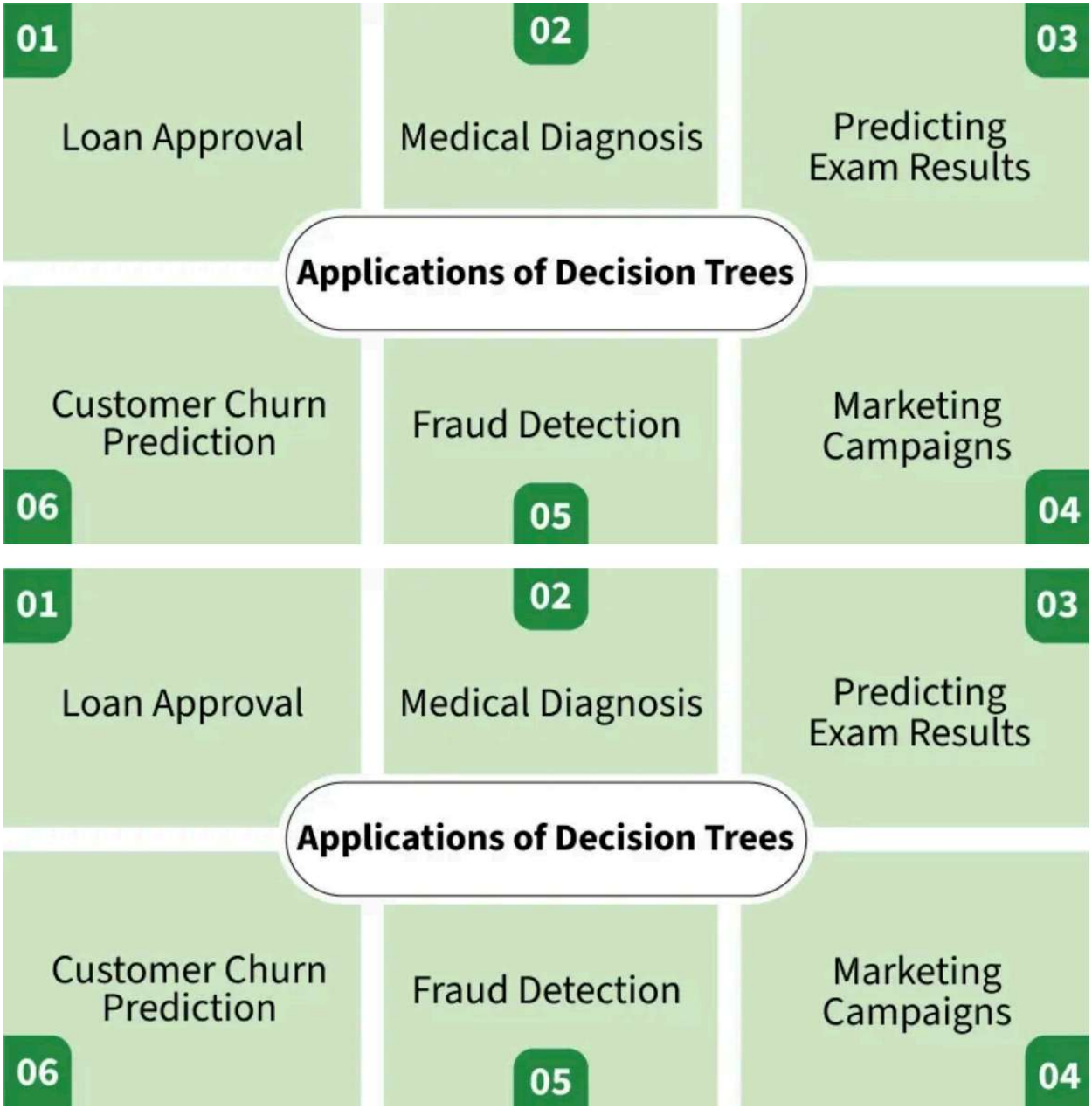
A Decision Tree learns a series of if-else questions about the data and chains them into a flowchart. It's one of the most interpretable ML models — you can draw it on paper and explain every decision.

A Decision Tree helps us to make decisions by mapping out different choices and their possible outcomes. It’s used in machine learning for tasks like classification and prediction.






Classification Tree
Email Spam ?
Yes
No
Yes/No decision icons.
Types of Decision Trees
Regression Tree
Area > 1000 sq ft
₹45L
₹45L
Numeric split at each node.


01
Loan Approval
02
Medical Diagnosis
03
Predicting Exam Results
Applications of Decision Trees
Customer Churn Prediction
06
Fraud Detection
05
Marketing Campaigns
04
01
Loan Approval
02
Medical Diagnosis
03
Predicting Exam Results
Applications of Decision Trees
Customer Churn Prediction
06
Fraud Detection
05
Marketing Campaigns
04

A Decision Tree helps us make decisions by showing different options and how they are related. It has a tree-like structure that starts with one main question called the root node which represents the entire dataset. From there, the tree branches out into different possibilities based on features in the data.

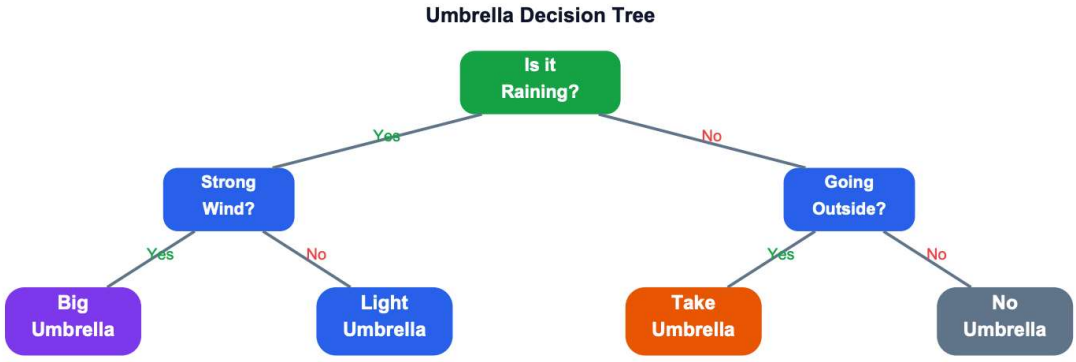
Root Node: Starting point representing the whole dataset.

Branches: Lines connecting nodes showing the flow from one decision to another.

Internal Nodes: Points where decisions are made based on data features.

Leaf Nodes: End points of the tree where the final decision or prediction is made.

Exxample for decision tree



Each internal node asks one clear yes/no question. Each leaf gives a final prediction.

The fastest way to understand a tree is to train one and visualise it:

```
 # ■■ Decision Tree: Fit, Inspect, Predict ■■
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
# Load iris dataset: 150 flowers, 4 features, 3 species
iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran
```

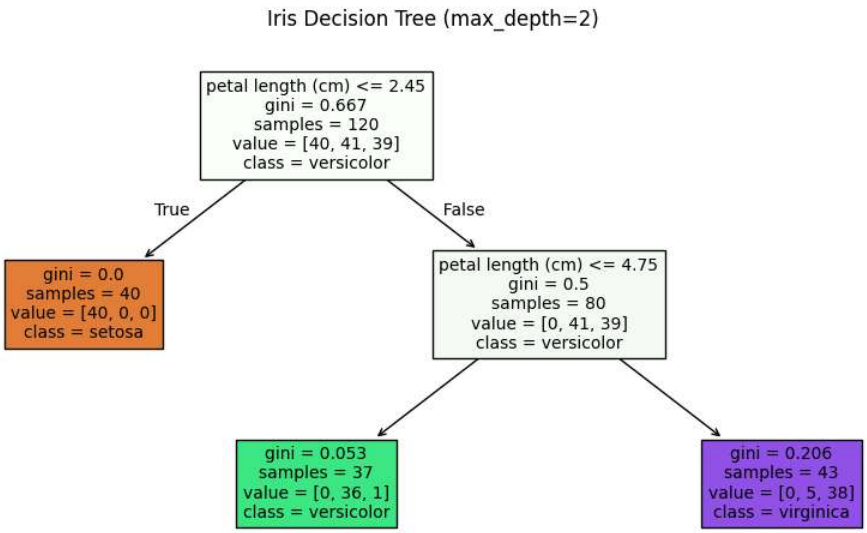
```
# ■■ Train a shallow tree (depth=2 so it's tiny and readable) ■■
tree = DecisionTreeClassifier(max_depth=2, random_state=42)
tree.fit(X_train, y_train)
print(f'Train accuracy: {tree.score(X_train, y_train):.2%}')
print(f'Test accuracy: {tree.score(X_test, y_test):.2%}')
```

```
Train accuracy: 95.00%
Test accuracy: 96.67%
```

```
# ■■ Text view: see the if-else rules as text ■■
print(export_text(tree, feature_names=iris.feature_names))
```

```
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) >  2.45
|   |--- petal length (cm) <= 4.75
|   |   |--- class: 1
|   |--- petal length (cm) >  4.75
|   |   |--- class: 2
```

```
# ■■ Visual view ■■
plt.figure(figsize=(10, 5))
plot_tree(tree, feature_names=iris.feature_names,
class_names=iris.target_names,
filled=True, rounded=False, fontsize=10)
plt.title('Iris Decision Tree (max_depth=2)'); plt.tight_layout(); plt.show(
```




```
# ■■ Predict a single new flower ■■
new_flower = [[5.1, 3.5, 1.4, 0.2]] # sepal_len, sepal_wid, petal_len, petal
pred = tree.predict(new_flower)[0]
print(f'Predicted species: {iris.target_names[pred]}')
print(f'Probabilities : {tree.predict_proba(new_flower)[0]}')
```

```
Predicted species: setosa
Probabilities : [1. 0. 0.]
```

How the Tree Chooses Questions — Gini Impurity

Gini Impurity is a metric used in Decision Trees to measure how mixed or impure the data inside a node is.

$$Gini(node) = 1 - \Sigma\ p_i^2 \text{ where } p\_i \text{ is the proportion of class } i$$

Which question creates the purest split?

The main goal of a Decision Tree is to create groups where most data points belong to a single class.

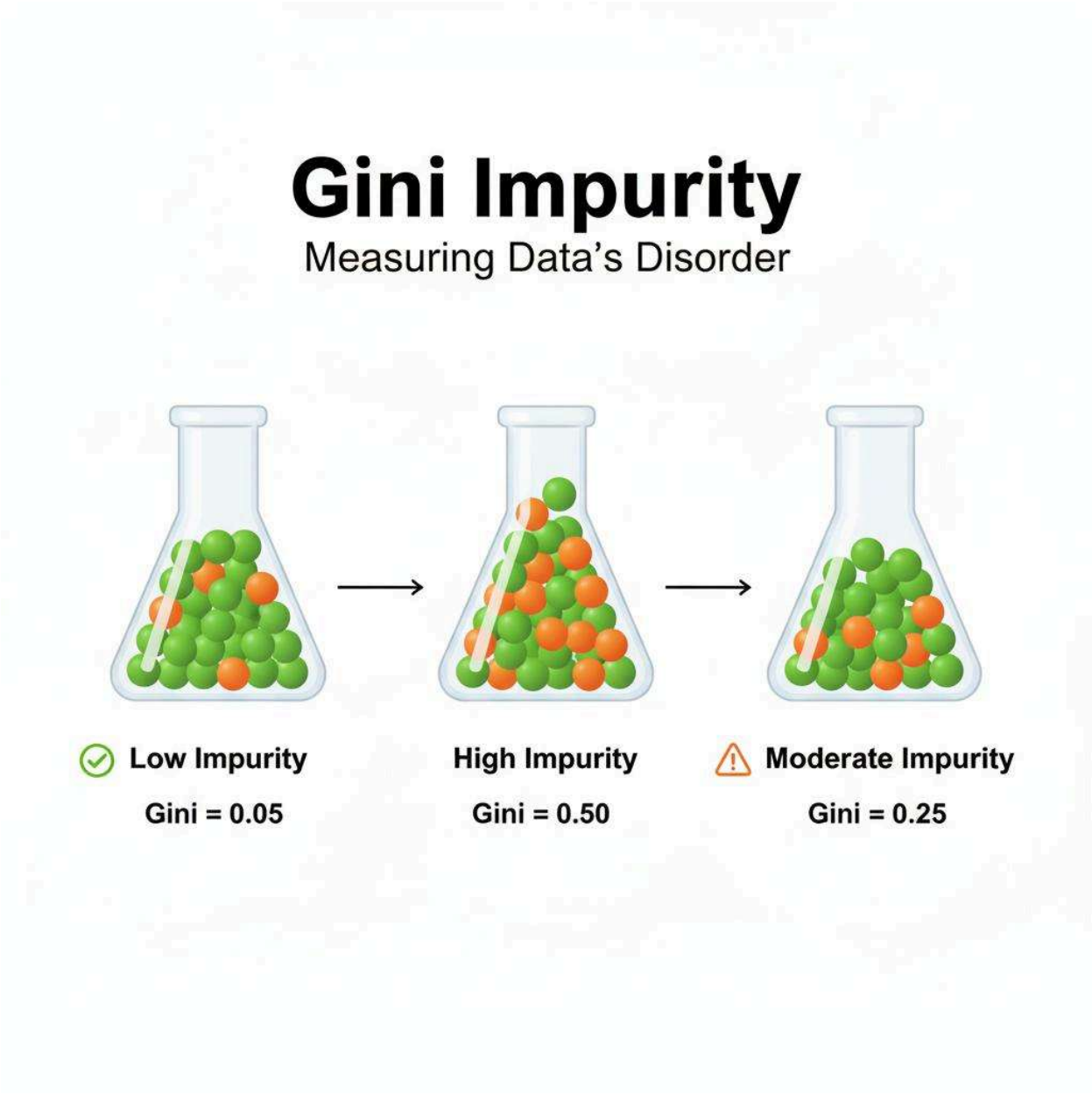
Lower Gini Impurity ⟶ More pure node ⟶ Better split

Higher Gini Impurity ⟶ More mixed node ⟶ Poor split

Gini Impurity Values

Gini Value	Meaning
0	Completely pure node
Near 0	Mostly pure
0.5	Maximum impurity in binary classification

Node Contents	Calculation	Gini Value	Interpretation
100% one class	(1 - (1.0^2))	**0.00**	Perfect purity — best split
50% / 50% split	(1 - (0.5^2 + 0.5^2))	**0.50**	Maximum impurity — worst split
70% class A, 30% class B	(1 - (0.7^2 + 0.3^2))	**0.42**	Fairly impure
90% class A, 10% class B	(1 - (0.9^2 + 0.1^2))	**0.18**	Mostly pure — good split



Double-click (or enter) to edit

Calculate Gini manually first, then let sklearn demonstrate the same logic:

```
# ■■ Gini Impurity: Manual + sklearn ■■
import numpy as np
from sklearn.tree import DecisionTreeClassifier
import matplotlib.pyplot as plt

# ■■ Part 1: Manual Gini calculation ■■
def gini_impurity(labels):
    """Calculate Gini impurity for a list of class labels."""
    labels = np.array(labels)
    n = len(labels)
    if n == 0: return 0
    classes = np.unique(labels)
    gini = 1.0
    for c in classes:
```

```
            p = np.sum(labels == c) / n # proportion of class c
            gini -= p ** 2
        return gini
```

```
# Test different node compositions
print('Pure node (all same) :', gini_impurity([1,1,1,1,1])) # 0.00
print('50-50 split :', gini_impurity([0,0,0,1,1,1])) # 0.50
print('70-30 split :', gini_impurity([0,0,0,0,0,0,0,1,1,1])) # 0.42
print('90-10 split :', gini_impurity([0]*9 + [1])) # 0.18
```

```
Pure node (all same) : 0.0
50-50 split : 0.5
70-30 split : 0.42000000000000004
90-10 split : 0.17999999999999994
```

```
# ■■ Part 2: Weighted Gini for a split ■■
def weighted_gini(left_labels, right_labels):
    """Gini after a split — weighted by group size."""
    n_total = len(left_labels) + len(right_labels)
    w_left = len(left_labels) / n_total
    w_right = len(right_labels) / n_total
    return w_left * gini_impurity(left_labels) + w_right * gini_impurity(rig
```

```
# Two possible splits on a feature — which is better?
# Split A: Glucose <= 120 vs Glucose > 120
left_A = [0,0,0,0,1] # Glucose <= 120: 4 healthy, 1 diabetic
right_A = [0,1,1,1,1] # Glucose > 120: 1 healthy, 4 diabetic
```

```
# Split B: Age <= 30 vs Age > 30
left_B = [0,0,1,1,0] # Age <= 30: mixed
right_B = [0,1,0,1,1] # Age > 30: mixed
```

```
print(f'Weighted Gini — Glucose split: {weighted_gini(left_A, right_A):.3f}'
print(f'Weighted Gini — Age split : {weighted_gini(left_B, right_B):.3f}')
print('Tree picks the split with LOWER weighted Gini → Glucose split is bett
```

```
Weighted Gini — Glucose split: 0.320
Weighted Gini — Age split : 0.480
Tree picks the split with LOWER weighted Gini → Glucose split is better!
```

```
# ■■ Part 3: See sklearn report the Gini per node ■■
from sklearn.datasets import make_classification
from sklearn.tree import export_text
X, y = make_classification(n_samples=200, n_features=4, random_state=42)
tree = DecisionTreeClassifier(max_depth=2, random_state=42)
tree.fit(X, y)
# export_text shows 'gini' value at each node
print(export_text(tree, feature_names=['F1','F2','F3','F4']))
# Each node shows: feature threshold | gini | samples | distribution
```

```
|--- F1 <= -0.15
|   |--- F3 <= -0.44
```

```
|   |   |--- class: 0
|   |--- F3 >  -0.44
|   |   |--- class: 0
|--- F1 >  -0.15
|   |--- F3 <= 0.63
|   |   |--- class: 1
|   |--- F3 >  0.63
|   |   |--- class: 0
```

Overfitting vs Underfitting

The most important concept in all of Machine Learning. Think of it as the studying for an exam problem:

• Underfitting: You barely studied. Model too simple, misses the real pattern.

• Overfitting: You memorised every practice question word-for-word. Fails on the real exam.

• Just Right: You understood the concepts — generalises to new questions.

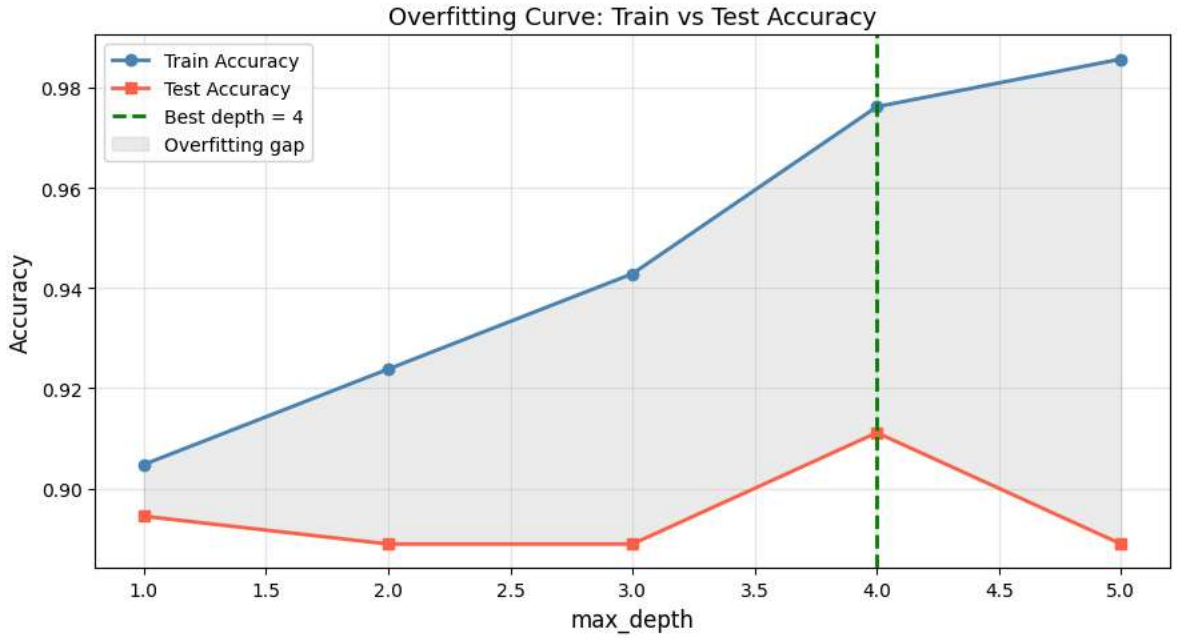
State	Train Accuracy	Test Accuracy	Diagnosis
Underfitting	Low (60%)	Low (58%)	Model is too simple and unable to learn patterns properly → inc
Just Right	High (88%)	High (85%)	Good model performance → small gap between train and test a
Overfitting	Very High (99%)	Low (64%)	Model memorized training data instead of learning general patt

Depth Tuning Loop: Find the Sweet Spot

Plot train vs test accuracy for every depth from 1 to 20. The sweet spot is where they diverge:

```
# ■■ Depth Tuning: Visualise the Overfitting Curve ■■
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
X, y = make_classification(n_samples=600, n_features=10, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, ran
depths = range(1, 6)
train_acc, test_acc = [], []
```

```
for d in depths:
    tree = DecisionTreeClassifier(max_depth=d, random_state=42)
    tree.fit(X_train, y_train)
    train_acc.append(accuracy_score(y_train, tree.predict(X_train)))
    test_acc.append(accuracy_score(y_test, tree.predict(X_test)))
best_depth = depths[test_acc.index(max(test_acc))]
plt.figure(figsize=(9, 5))
plt.plot(depths, train_acc, 'o-', color='steelblue', lw=2, label='Train Accu
plt.plot(depths, test_acc, 's-', color='tomato', lw=2, label='Test Accuracy'
plt.axvline(x=best_depth, color='green', linestyle='--', lw=2,
            label=f'Best depth = {best_depth}')
plt.fill_between(depths,
                 train_acc, test_acc,
                 alpha=0.15, color='gray', label='Overfitting gap')
plt.xlabel('max_depth', fontsize=12)
plt.ylabel('Accuracy', fontsize=12)
plt.title('Overfitting Curve: Train vs Test Accuracy', fontsize=13)
plt.legend(); plt.grid(alpha=0.3); plt.tight_layout(); plt.show()

# INSIGHT: Train acc always increases → tree memorises more
# Test acc peaks then falls → past the peak = overfitting
print(f'Best test accuracy {max(test_acc):.2%} at depth={best_depth}')
```




```
Best test accuracy 91.11% at depth=4
```

Other Anti-Overfitting Parameters

max_depth is not the only tool — here are the other key parameters:

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score
import numpy as np
# Cross-validation: more reliable than a single train/test split
# (Uses 5 different splits and averages the score)

configs = [
{'max_depth': None}, # Unrestricted (likely overfit)
{'max_depth': 5}, # Limit depth
{'max_depth': 5, 'min_samples_split': 10}, # Split only if 10+ samples
{'max_depth': 5, 'min_samples_leaf': 5}, # Each leaf needs 5+ samples
{'max_depth': 5, 'max_features': 'sqrt'}, # Only consider sqrt(n_features) p
]

print(f'{'Config':45s} {'CV Score (mean±std)':20s}')
print('-' * 60)


for cfg in configs:
  tree = DecisionTreeClassifier(**cfg, random_state=42)
  scores = cross_val_score(tree, X, y, cv=7, scoring='accuracy')
  print(f'{str(cfg):45s} {scores.mean():.3f} ± {scores.std():.3f}')




# min_samples_leaf is often the most effective single parameter to reduce ov
```

```
Config                                        CV Score (mean±std)
------------------------------------------------------------
{'max_depth': None}                           0.892 ± 0.057
{'max_depth': 5}                              0.891 ± 0.046
{'max_depth': 5, 'min_samples_split': 10}     0.894 ± 0.056
{'max_depth': 5, 'min_samples_leaf': 5}       0.901 ± 0.046
{'max_depth': 5, 'max_features': 'sqrt'}      0.887 ± 0.056
```

RANDOM FOREST

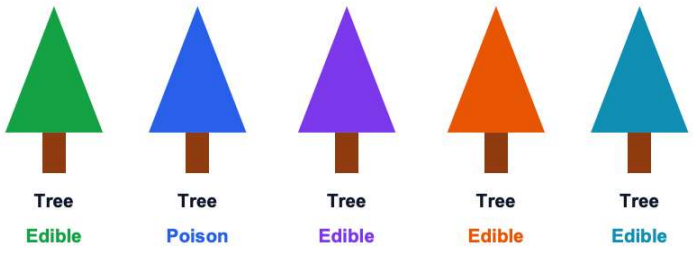
A random forest is an ensemble learning method that combines the predictions from multiple decision trees to produce a more accurate and stable prediction. It can be used for both classification and regression tasks.

One tree can overfit badly. The solution: build many trees on random subsets of data and features, then let them vote. Each tree makes different errors — majority voting cancels individual mistakes

Two sources of randomness make trees diverse:

• Bootstrap Sampling: Each tree trains on a random sample (with replacement) of rows — about 63% unique rows, 37% left out ('out-of-bag' samples for free validation).

• Random Feature Subset: At each split, only √(n_features) features are considered. This forces trees to be different from each other.



5 trees trained on different samples, using different features → diverse errors → voting corrects most.

Tree	Prediction
Tree 1	Edible
Tree 2	Posion
Tree 3	Edible
Tree 4	Edible
Tree 5	Edible

OOB Score, Voting, and n_estimators Sweep

Three demonstrations: OOB validation, manual voting insight, and the n_estimators trade-off:

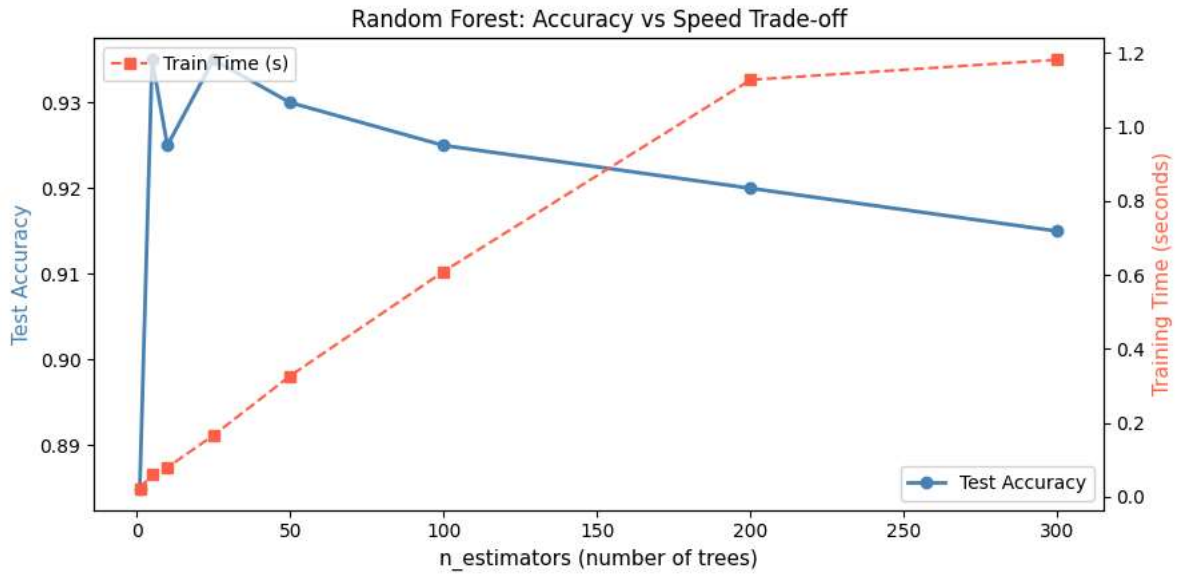
```
# ■■ OOB Score + n_estimators Sweep ■■
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import numpy as np, matplotlib.pyplot as plt, time

X, y = make_classification(n_samples=1000, n_features=15, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran
```

```
# ■■ Demo 1: OOB Score (free validation from bootstrap) ■■
rf_oob = RandomForestClassifier(n_estimators=10, oob_score=True, random_stat
rf_oob.fit(X_train, y_train)
print(f'OOB Score : {rf_oob.oob_score_:.2%} ← estimated on ~37% unseen rows
print(f'Test Score : {rf_oob.score(X_test, y_test):.2%} ← actual held-out te
print('OOB score is a free estimate — no need for a separate validation set!
```

```
OOB Score : 71.50% ← estimated on ~37% unseen rows per tree
Test Score : 75.32% ← actual held-out test
```

```
OOB score is a free estimate — no need for a separate validation set!
/usr/local/lib/python3.12/dist-packages/sklearn/ensemble/_forest.py:612: User
  warn(
```

```
# ■■ Demo 2: How n_estimators affects accuracy and speed ■■
n_trees_list = [1, 5, 10, 25, 50, 100, 200, 300]
accs, times = [], []
for n in n_trees_list:
  t0 = time.time()
  rf = RandomForestClassifier(n_estimators=n, random_state=42, n_jobs=-1)
  rf.fit(X_train, y_train)
  accs.append(accuracy_score(y_test, rf.predict(X_test)))
  times.append(time.time() - t0)
fig, ax1 = plt.subplots(figsize=(9, 4.5))
ax2 = ax1.twinx()
ax1.plot(n_trees_list, accs, 'o-', color='steelblue', lw=2, label='Test Accu
ax2.plot(n_trees_list, times, 's--', color='tomato', lw=1.5, label='Train Ti
ax1.set_xlabel('n_estimators (number of trees)', fontsize=11)
ax1.set_ylabel('Test Accuracy', color='steelblue', fontsize=11)
ax2.set_ylabel('Training Time (seconds)', color='tomato', fontsize=11)
ax1.set_title('Random Forest: Accuracy vs Speed Trade-off')
ax1.legend(loc='lower right'); ax2.legend(loc='upper left')
plt.tight_layout(); plt.show()
# INSIGHT: Accuracy stabilises quickly (~50-100 trees); time grows linearly
# There's a 'sweet spot' — more trees don't help much but cost more time
```



Feature Importance

Random Forest measures how much each feature reduces Gini impurity across all trees and all splits. This gives a feature importance score (0–1, sums to 1.0) — a ranked explanation of what drives predictions.

Feature Importance: Plot, Interpret, and Select

```
# ■■ Feature Importance: Plot + Feature Selection ■■
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_diabetes # regression dataset repurposed a
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelBinarizer
import pandas as pd, numpy as np, matplotlib.pyplot as plt
```

```
# Use the diabetes (Pima Indians) dataset from before
import pandas as pd
# Download the dataset if it's not present
import os
if not os.path.exists('diabetes.csv'):
    !wget https://raw.githubusercontent.com/plotly/datasets/master/diabetes.
df = pd.read_csv('diabetes.csv')
fix_cols = ['Glucose','BloodPressure','SkinThickness','Insulin','BMI']
for col in fix_cols: df[col] = df[col].replace(0, df[col].median())
X = df.drop('Outcome', axis=1)
y = df['Outcome']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran
rf = RandomForestClassifier(n_estimators=200, random_state=42)
rf.fit(X_train, y_train)
```

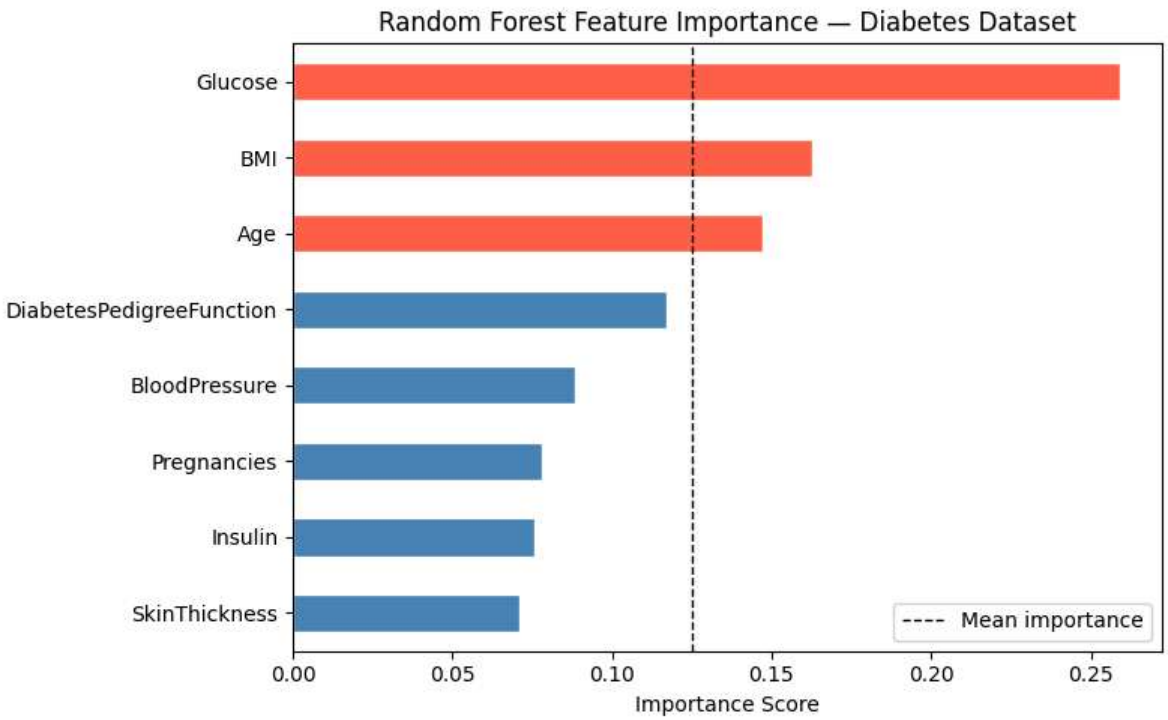
```
▾ RandomForestClassifier
RandomForestClassifier(n_estimators=200, random_state=42)
```

```
# ■■ Feature importances ■■
imp = pd.Series(rf.feature_importances_, index=X.columns).sort_values(ascend
print('Feature Importances (higher = more important):')
print(imp.round(3))
```

```
Feature Importances (higher = more important):
Glucose                     0.259372
BMI                         0.162542
Age                         0.147397
DiabetesPedigreeFunction    0.117283
BloodPressure               0.088270
Pregnancies                 0.078288
Insulin                     0.075817
SkinThickness               0.071030
dtype: float64
```

```
# ■■ Bar chart ■■
imp_sorted = imp.sort_values() # ascending for barh
```

```
colors = ['tomato' if g > imp.mean() else 'steelblue' for g in imp_sorted]
imp_sorted.plot(kind='barh', color=colors, figsize=(8,5), edgecolor='white')
plt.axvline(x=imp.mean(), color='black', linestyle='--', linewidth=1, label=
plt.title('Random Forest Feature Importance — Diabetes Dataset', fontsize=12
plt.xlabel('Importance Score'); plt.legend(); plt.tight_layout(); plt.show()
```




```
# ■■ Feature Selection: keep only top features ■■
top_features = imp[imp > imp.mean()].index.tolist()
print(f'\nTop features (above average): {top_features}')
rf_top = RandomForestClassifier(n_estimators=200, random_state=42)
rf_top.fit(X_train[top_features], y_train)
full_acc = rf.score(X_test, y_test)
top_acc = rf_top.score(X_test[top_features], y_test)
print(f'Accuracy with all {len(X.columns)} features : {full_acc:.2%}')
print(f'Accuracy with top {len(top_features)} features : {top_acc:.2%}')
print('Often comparable — low-importance features can add noise!')
```

```
Top features (above average): ['Glucose', 'BMI', 'Age']
Accuracy with all 8 features : 77.27%
Accuracy with top 3 features : 72.73%
Often comparable — low-importance features can add noise!
```

LAB: Classify Mushrooms — Edible or Poisonous

8,124 mushrooms, 22 categorical features (cap shape, odor, gill color...). Target: class = 'e' (edible) or 'p' (poisonous). All features are text — we must encode them to numbers first.

Step 1 — Load, Encode, Split

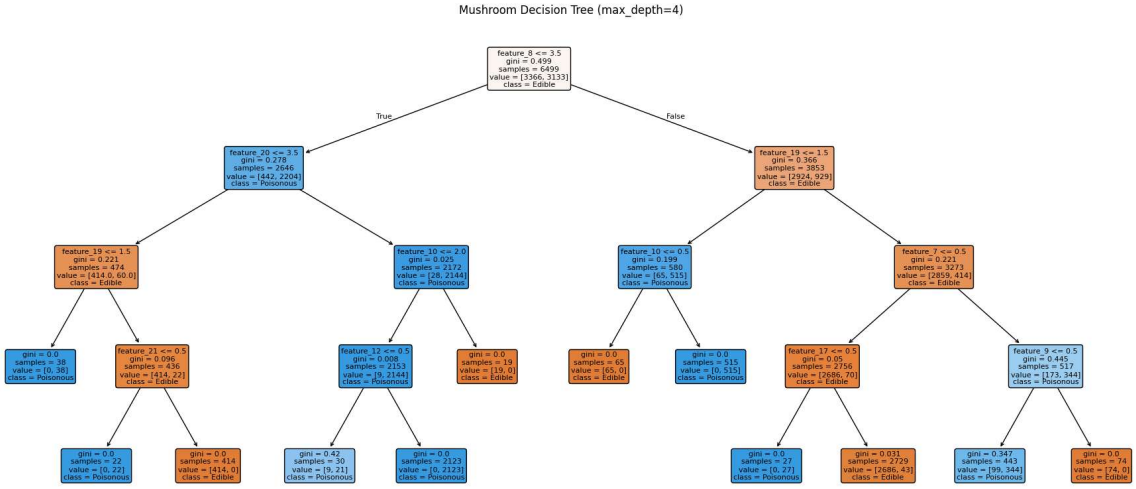
```
# ■■ Load and Encode ■■
import pandas as pd, numpy as np, matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score

# Download the dataset if it's not present
import os
if not os.path.exists('mushrooms.csv'):
    !wget https://archive.ics.uci.edu/ml/machine-learning-databases/mushroom

df = pd.read_csv('mushrooms.csv') # kaggle: 'mushroom classification'
# The downloaded dataset does not have headers, so we need to add them or ha
# For simplicity, let's load it and assume the first column is 'class' for n
# In a real scenario, you would need to define column names.

# For the mushroom dataset from UCI, the 'class' column is the first one.
# The column names are not in the CSV, so we'll need to handle it accordingl
# Let's assume the dataset from UCI is comma-separated without header.
# The 'class' column is the first column in this dataset.

# Let's reload with proper column names if available, or use integers.
# Based on common usage of this dataset, let's add dummy column names
# and then assign the first column as 'class'.
# There are 23 columns. The first one is the class.
column_names = ['class'] + [f'feature_{i}' for i in range(22)]
df = pd.read_csv('mushrooms.csv', header=None, names=column_names)

print(df.shape) # (8124, 23)
print(df['class'].value_counts()) # p=4208 poisonous, e=3916 edible

# Encode every column: text → integer
# LabelEncoder: 'e'→0, 'p'→1 (and similarly for all features)
le = LabelEncoder()
df_enc = df.copy()
for col in df_enc.columns:
    df_enc[col] = le.fit_transform(df_enc[col])
X = df_enc.drop('class', axis=1) # 22 features
y = df_enc['class'] # 0=edible, 1=poisonous
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
print(f'Train: {len(X_train)} rows | Test: {len(X_test)} rows')
```

```
--2026-05-29 12:32:54--  https://archive.ics.uci.edu/ml/machine-learning-data
Resolving archive.ics.uci.edu (archive.ics.uci.edu)... 128.195.10.252
Connecting to archive.ics.uci.edu (archive.ics.uci.edu)|128.195.10.252|:443..
HTTP request sent, awaiting response... 200 OK
Length: unspecified
Saving to: ‘mushrooms.csv’

mushrooms.csv           [ <=>                ] 364.95K  --.-KB/s    in 0.05s

2026-05-29 12:32:54 (7.49 MB/s) - ‘mushrooms.csv’ saved [373704]

(8124, 23)
class
e    4208
p    3916
Name: count, dtype: int64
Train: 6499 rows | Test: 1625 rows
```

Step 2 — Train a Decision Tree + Visualise

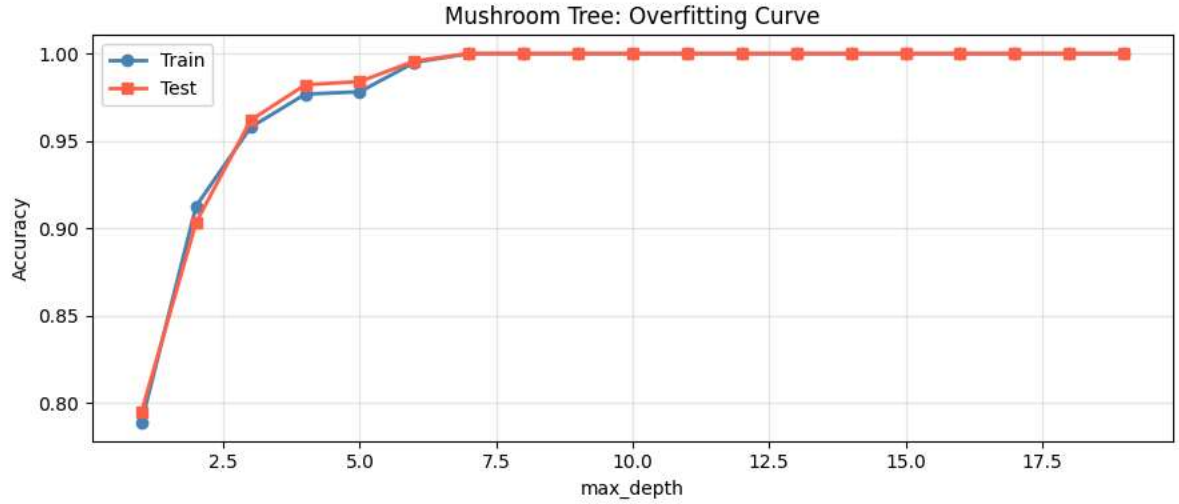
```
# ■■ Decision Tree: Train + Visualise ■■
# ■■ Shallow tree (depth=4) — interpretable ■■
dt = DecisionTreeClassifier(max_depth=4, random_state=42)
dt.fit(X_train, y_train)
print(f'Decision Tree (depth=4) accuracy: {dt.score(X_test, y_test):.2%}')
# Text rules — read them like an if-else chain
print(export_text(dt, feature_names=list(X.columns)))
# ■■ Find the single most important feature ■■
imp = pd.Series(dt.feature_importances_, index=X.columns)
top = imp.idxmax()
print(f'Most important feature: {top} (importance={imp[top]:.3f})')
# Hint: 'odor' almost always comes first!
# odor=none → almost certainly edible. Smell saves lives.
# ■■ Visual tree ■■
plt.figure(figsize=(18, 8))
plot_tree(dt, feature_names=X.columns, class_names=['Edible','Poisonous'],
filled=True, rounded=True, fontsize=8) # Changed fontsize to an integer
plt.title('Mushroom Decision Tree (max_depth=4)'); plt.tight_layout()
plt.savefig('mushroom_tree.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
Decision Tree (depth=4) accuracy: 98.22%
|--- feature_8 <= 3.50
|   |--- feature_20 <= 3.50
|   |   |--- feature_19 <= 1.50
|   |   |   |--- class: 1
|   |   |--- feature_19 >  1.50
|   |   |   |--- feature_21 <= 0.50
|   |   |   |   |--- class: 1
|   |   |   |--- feature_21 >  0.50
|   |   |   |   |--- class: 0
|   |--- feature_20 >  3.50
|   |   |--- feature_10 <= 2.00
|   |   |   |--- feature_12 <= 0.50
|   |   |   |   |--- class: 1
|   |   |   |--- feature_12 >  0.50
|   |   |   |   |--- class: 1
|   |   |--- feature_10 >  2.00
|   |   |   |--- class: 0
|--- feature_8 >  3.50
|   |--- feature_19 <= 1.50
|   |   |--- feature_10 <= 0.50
|   |   |   |--- class: 0
|   |   |--- feature_10 >  0.50
|   |   |   |--- class: 1
|   |--- feature_19 >  1.50
|   |   |--- feature_7 <= 0.50
|   |   |   |--- feature_17 <= 0.50
|   |   |   |   |--- class: 1
|   |   |   |--- feature_17 >  0.50
|   |   |   |   |--- class: 0
|   |   |--- feature_7 >  0.50
|   |   |   |--- feature_9 <= 0.50
|   |   |   |   |--- class: 1
|   |   |   |--- feature_9 >  0.50
|   |   |   |   |--- class: 0

Most important feature: feature_8 (importance=0.367)
```



Step 3 — Depth Sweep: Find the Overfit Point

```
# ■■ Depth Sweep ■■
train_acc, test_acc = [], []
for d in range(1, 20):
    tree = DecisionTreeClassifier(max_depth=d, random_state=42)
    tree.fit(X_train, y_train)
    train_acc.append(tree.score(X_train, y_train))
    test_acc.append(tree.score(X_test, y_test))
plt.figure(figsize=(9, 4))
plt.plot(range(1,20), train_acc, 'o-', lw=2, label='Train', color='steelblue
plt.plot(range(1,20), test_acc, 's-', lw=2, label='Test', color='tomato')
plt.xlabel('max_depth'); plt.ylabel('Accuracy')
plt.title('Mushroom Tree: Overfitting Curve')
plt.legend(); plt.grid(alpha=0.3); plt.tight_layout(); plt.show()
best = range(1,20)[test_acc.index(max(test_acc))]
print(f'Best depth: {best}, Test accuracy: {max(test_acc):.2%}')
```




```
Best depth: 7, Test accuracy: 100.00%
```

Step 4 — Random Forest: Boost Accuracy + Full Importance Chart

```
# ■■ Random Forest: Full Evaluation + Ablation Test ■■
rf = RandomForestClassifier(n_estimators=100, oob_score=True, random_state=4
rf.fit(X_train, y_train)
print(f'Random Forest Test Accuracy: {rf.score(X_test, y_test):.2%}')
print(f'Random Forest OOB Accuracy: {rf.oob_score_:.2%}')
print(classification_report(y_test, rf.predict(X_test), target_names=['Edibl
# Cross-validation for robustness check (5 different splits)
cv_scores = cross_val_score(rf, X, y, cv=5, scoring='accuracy')
print(f'5-Fold CV: {cv_scores.mean():.2%} ± {cv_scores.std():.2%}')
# Feature importance bar chart
rf_imp = pd.Series(rf.feature_importances_, index=X.columns).sort_values(asc
rf_imp.plot(kind='barh', figsize=(9, 8), color='mediumseagreen', edgecolor='
```

```
plt.title('Random Forest Feature Importance — Mushroom Dataset', fontsize=12
plt.xlabel('Importance Score'); plt.tight_layout(); plt.show()
# EXPERIMENT: Remove 'odor' and retrain — does accuracy drop dramatically?
X_no_odor = X.drop(columns=['feature_8']) # Changed 'odor' to 'feature_8'
X_tr2, X_te2, ytr, yte = train_test_split(X_no_odor, y, test_size=0.2, randc
rf2 = RandomForestClassifier(n_estimators=100, random_state=42).fit(X_tr2, y
print(f'\nAccuracy WITHOUT odor feature: {rf2.score(X_te2, yte):.2%}')
print(f'Accuracy WITH odor feature: {rf.score(X_test, y_test):.2%}')
```

```
Random Forest Test Accuracy: 100.00%
Random Forest OOB Accuracy: 100.00%
              precision    recall  f1-score   support

      Edible       1.00      1.00      1.00       842
   Poisonous       1.00      1.00      1.00       783

    accuracy                           1.00      1625
   macro avg       1.00      1.00      1.00      1625
weighted avg       1.00      1.00      1.00      1625
```

Step 5 — Predict for a New Mushroom

```
# ■■ Predict New Mushrooms ■■
# Grab a single mushroom from the test set to predict
sample_index = 0
sample = X_test.iloc[[sample_index]]
true_label = y_test.iloc[sample_index]
pred = rf.predict(sample)[0]
prob = rf.predict_proba(sample)[0]
label = 'Edible' if pred == 0 else 'POISONOUS'
actual = 'Edible' if true_label == 0 else 'POISONOUS'
print(f'Prediction : {label}')
print(f'Actual : {actual}')
print(f'Confidence : {max(prob):.1%}')
print(f'Correct? : {'Yes!' if pred == true_label else 'No...'}')
# Try multiple samples
for i in range(10):
    samp = X_test.iloc[[i]]
    pred = rf.predict(samp)[0]
    conf = max(rf.predict_proba(samp)[0])
    true = y_test.iloc[i]
    mark = '✓' if pred == true else 'X'
```